

DATA STRUCTURES AND ALGORITHMS-3

25CS2103E

MODULE-2

Topic: String Algorithms

Naive and KMP String Pattern Matching Algorithms

String Matching Problem

- String matching also called pattern matching is one of the most fundamental problems in Computer Science.
- It is widely used in text processing, search engines, bioinformatics, plagiarism detection, data mining, and cybersecurity.
- The objective is to determine whether a pattern occurs inside a larger text, and if it does, identify all its occurrences.

String Matching Problem

Definition:- Given a text T of length n and a pattern P of length m (where $m \leq n$), find all positions (shifts) in T where P occurs. We say the pattern P occurs with shift s in the text T if $0 \leq s \leq n - m$ and $T[s + 1 .. s + m] = P[1 .. m]$. Such a shift s is called a valid shift; otherwise it is an invalid shift.

Text (T): The large string being searched, length n .

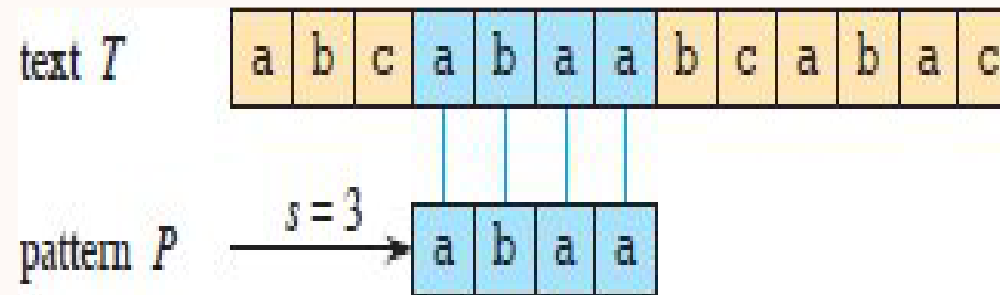
Pattern (P): The target substring, length m .

Goal: Find all valid shifts s where $T[s+1 .. s+m] = P[1 .. m]$.

The string-matching problem is the problem of finding all valid shifts with which a given pattern P occurs in a given text T .

Eg: Find pattern $P = \mathbf{abaa}$ in the text $T = \mathbf{abcabaabcabac}$.

The pattern occurs only once in the text, at shift $s = 3$, which is a valid shift.



Example

Given T =

A	B	A	B	C	A	B	D	A	B	C	A	A	B	C
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

and P = ABC

- **The first occurrence starts at T[2]**
- **The second occurrence starts at T[8]**
- **The third occurrence starts at T[12]**

Naive String Pattern Matching Algorithm

- The naive string matching algorithm is the simplest, brute-force method for finding all occurrences of a specific pattern within a larger text.
- It does not use any information from previous comparisons, simply shifting the pattern by one character after a mismatch or a successful match.
- Like brute force, it also checks for all characters of the main string in order to find the pattern. Hence, its time complexity is **$O(mn)$** where the 'm' is the size of pattern and 'n' is the size of the main string. This algorithm is helpful for smaller texts only.
- The naive pattern searching algorithm does not require any pre-processing phases.
- It does not occupy extra space to perform the operation.

Text: a b c d a b f a b e g

Pattern: abe

Text	a b c d a b f a b e g	No of comparisons
1 st iteration	a b e	3
2 nd	a b e	1

Text: a b c d a b f a b e g

Pattern: abe

Text	a b c d a b f a b e g	No of comparisons
1 st iteration	a b e	3
2 nd	a b e	1
3 rd	a b e	1
4 th	a b e	1
5 th	a b e	3
6 th	a b e	1
7 th	a b e	1
8 th	a b e	3

Result in 8th iteration and the number of comparisons are 14

Let's go through an example with $T=ababcbabdabcaabc$ and $P=abc$.

1. Text: $ababcbabdabcaabc$

2. Pattern: abc

The algorithm checks for the pattern at every position in the text:

- At position 0: $T[0..2] = aba$ (no match)
- At position 1: $T[1..3] = bab$ (no match)
- At position 2: $T[2..4] = abc$ (match found, index 2)
- At position 3: $T[3..5] = bca$ (no match)
- At position 4: $T[4..6] = cab$ (no match)
- At position 5: $T[5..7] = abd$ (no match)
- At position 6: $T[6..8] = bda$ (no match)
- At position 7: $T[7..9] = dab$ (no match)
- At position 8: $T[8..10] = abc$ (match found, index 8)
- At position 9: $T[9..11] = bca$ (no match)
- At position 10: $T[10..12] = caa$ (no match)
- At position 11: $T[11..13] = aab$ (no match)
- At position 12: $T[12..14] = abc$ (match found, index 12)

DSA-3 The matches are found in indices **2, 8, and 12**.

Naïve Algorithm

Algorithm naïve_string(Text, Pattern)

begin

 n=length(Text)

 m=length(Pattern)

for i=0 to n-m step 1 **do**

for j=0 to m-1 step 1 **do**

if Text[i+j]≠ Pattern[j]:
 break

end if

end for

if j==m **then**

 print "Pattern found at index", i

end if

end for

stop

Text (index i)

A	B	A	B	C
0	1	2	3	4

Pattern (index j)

A B C

//Naive string pattern matching program

```
public class Naivestring {
    public static void search(String text, String pat) {
        int m=text.length();
        int m=pat.length();
        int i,j;
        for (i=0;i<=n-m;i++)
        {
            for (j=0;j<m;j++)
            {
                if (text.charAt(i+j)!=pat.charAt(j))
                    break;
            }
            if (j==m)
                System.out.println("Pattern found at index " +i);
        }
    }
}
```

```
public static void main(String[] args) {
    // TODO Auto-generated method stub
    String text1 = "AABAACAADAABAABA";
    String pat1 = "AABA";
    System.out.println("First string output " + text1+"
"+pat1);
    search(text1,pat1);
    String text2 = "ababcabdabcaabc";
    String pat2 = "abc";
    System.out.println("Second string output " + text2+"
"+pat2);
    search(text2,pat2);
}
OUTPUT:
```

First string output

AABAACAADAABAABA AABA

Pattern found at index 0

Pattern found at index 9

Pattern found at index 12

Second string output ababcabdabcaabc abc

Pattern found at index 2

Pattern found at index 8

Pattern found at index 12

Consider the worst case: $P = \text{"aaab"}$ style patterns against runs of the same character.

Text $T = \text{"aaaaaaaaab"}$ ($n=10$), keyword $P = \text{"aaab"}$ ($m=4$). Windows $i = 0 \dots 6$:

$i=0$: a a a b vs $T[0..3]=\text{a a a a} \rightarrow a=a, a=a, a=a, b \neq a \rightarrow 4 \text{ comparisons, FAIL}$

$i=1$: a a a b vs $T[1..4]=\text{a a a a} \rightarrow a=a, a=a, a=a, b \neq a \rightarrow 4 \text{ comparisons, FAIL}$

$i=2$: a a a b vs $T[2..5]=\text{a a a a} \rightarrow 4 \text{ comparisons, FAIL}$

$i=3$: a a a b vs $T[3..6]=\text{a a a a} \rightarrow 4 \text{ comparisons, FAIL}$

$i=4$: a a a b vs $T[4..7]=\text{a a a a} \rightarrow 4 \text{ comparisons, FAIL}$

$i=5$: a a a b vs $T[5..8]=\text{a a a a} \rightarrow 4 \text{ comparisons, FAIL}$

$i=6$: a a a b vs $T[6..9]=\text{a a a b} \rightarrow 4 \text{ comparisons, MATCH}$

Now look at what was redundant: the character $T[3]='a'$ alone was compared four separate times, at $i=0$, then again at $i=1$, $i=2$, $i=3$.

- ✓ Naïve pattern matching is simple and correct but it lets the text pointer backtrack after every mismatch and ignores the pattern's internal structure, causing the same text characters to be compared repeatedly.
- ✓ Its cost is $O(n \cdot m)$ in the worst case, triggered precisely by the repetitive, partially-matching keyword text.
- ✓ The repeated comparisons are the redundant re-examination of characters already inspected in earlier alignments and eliminating them is the whole motivation for KMP, Boyer–Moore, and Rabin–Karp.

Complexity Analysis

- **Time Complexity (Worst Case): $O(n*m)$.** This occurs in cases where the text and pattern have many repetitive characters (e.g., Text: AAAAAA, Pattern: AAA) or have near matches. For every possible starting position, nearly the entire pattern is compared.
- **Time Complexity (Best Case): $O(n)$.** This occurs when the first character of the pattern does not appear in the text at all, leading to an immediate mismatch at most positions.
- **Space Complexity: $O(1)$.** The algorithm requires a constant amount of extra memory regardless of the input size, as it only uses a few variables for indices and lengths.

Advantages

- Very simple to understand and implement.
- No preprocessing and no extra memory required ($O(1)$ space).
- Works well for short patterns, small texts, or when matches are rare.

Disadvantages

- Inefficient for large or highly repetitive inputs — up to $O(n \cdot m)$ time.
- Re-examines characters it has already compared, wasting work.
- Slower than smarter algorithms (KMP, Boyer–Moore, Rabin–Karp) that avoid redundant comparisons.

- Text searching
- Find and replace
- Spell checking
- DNA sequence analysis
- Plagiarism detection
- Log file analysis
- Source code search
- Database search
- Configuration file search
- Virus signature detection
- Educational demonstrations
- Simple command-line search utilities

Knuth-Morris-Pratt (KMP) algorithm

- The Knuth-Morris-Pratt (KMP) algorithm is a string pattern matching algorithm developed by Donald Knuth, James Morris and Vaughan Pratt. Hence the name Knuth-Morris-Pratt (KMP) Algorithm.
- The Knuth-Morris-Pratt (KMP) algorithm is a highly efficient string-searching method that searches for occurrences of a pattern within a text in linear $O(n + m)$ time.
- Unlike the naive brute-force method, which resets completely and backtracks the text pointer after a mismatch, KMP uses the structural information of the pattern itself to skip redundant comparisons.
- It achieves this by preprocessing the pattern string to construct a Failure Function.
- The failure function, denoted as π or $lps[]$ (Longest Proper Prefix which is also a Suffix), stores the length of the longest proper prefix of the pattern substring that is also a suffix of that substring.
- Proper Prefix: A prefix of a string that is not the entire string itself (e.g., proper prefixes of "abc" are "", "a", and "ab").
- Suffix: A concluding segment of a string (e.g., suffixes of "abc" are "", "c", "bc", and "abc").
- When a mismatch occurs at a specific index in the pattern, the failure function tells the algorithm exactly where to slide the pattern to resume matching, ensuring the text pointer never backtracks.

lps[] table for the pattern string: P = "ABABC"

Index (i)	Substring P[0...i]	Proper Prefixes	Suffixes	Longest Matching Length (lps[i])
0	"A"	None	"A"	0
1	"AB"	"A"	"B"	0
2	"ABA"	"A", "AB"	"A", "BA"	1 (Prefix "A" = Suffix "A")
3	"ABAB"	"A", "AB", "ABA"	"B", "AB", "BAB"	2 (Prefix "AB" = Suffix "AB")
4	"ABABC"	"A", "AB", "ABA", "ABAB"	"C", "BC", "ABC", "BABC"	0

Final lps array for "ABABC": [0, 0, 1, 2, 0]

Examples for creating an LPS array for different patterns

1 2 3 4 5 6 7 8 9 10
P1: a b c d a b e a b f

Examples for creating an LPS array for different patterns

	1	2	3	4	5	6	7	8	9	10
P1:	a	b	c	d	a	b	e	a	b	f
	0	0	0	0	1	2	0	1	2	0

Examples for creating an LPS array for different patterns

1 2 3 4 5 6 7 8 9 10

P1: a b c d a b e a b f

0 0 0 0 1 2 0 1 2 0

P2: a b c d e a b f a b c

Examples for creating an LPS array for different patterns

1 2 3 4 5 6 7 8 9 10

P1: a b c d a b e a b f

0 0 0 0 1 2 0 1 2 0

P2: a b c d e a b f a b c

0 0 0 0 0 1 2 0 1 2 3

P3: a a b c a d a a b e

0 1 0 0 1 0 1 2 3 0

P4: a a a a b a a c d

0 1 2 3 0 1 2 0 0

How KMP Algorithm works

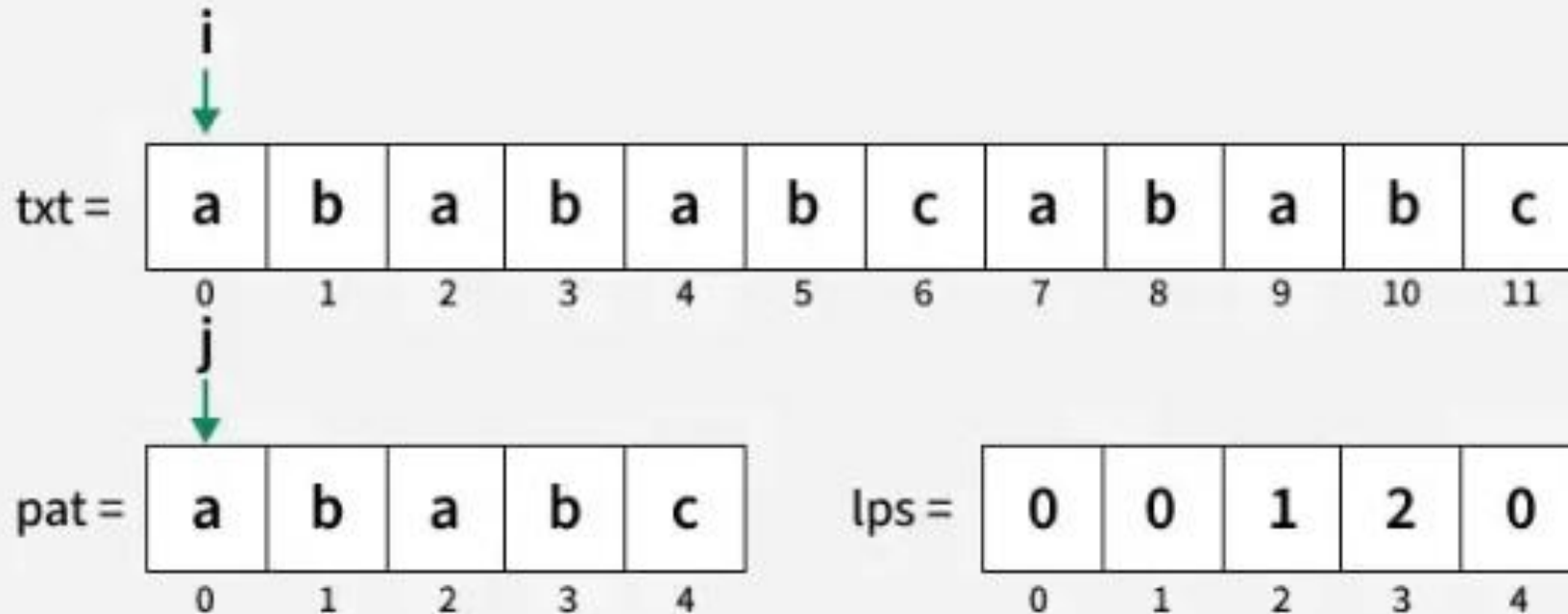
- Text : a b a b c a b c a b a b a b d
 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
- Pattern: a b a b d
- First, we will prepare the “ π ” Table (LPS[] array)

0	1	2	3	4
a	b	a	b	d
0	0	1	2	0

Working of KMP

01
Step

Initialize pointers at the beginning of both text and pattern.
The lps for the given pattern would be $\text{lps}[] = \{0, 0, 1, 2, 0\}$

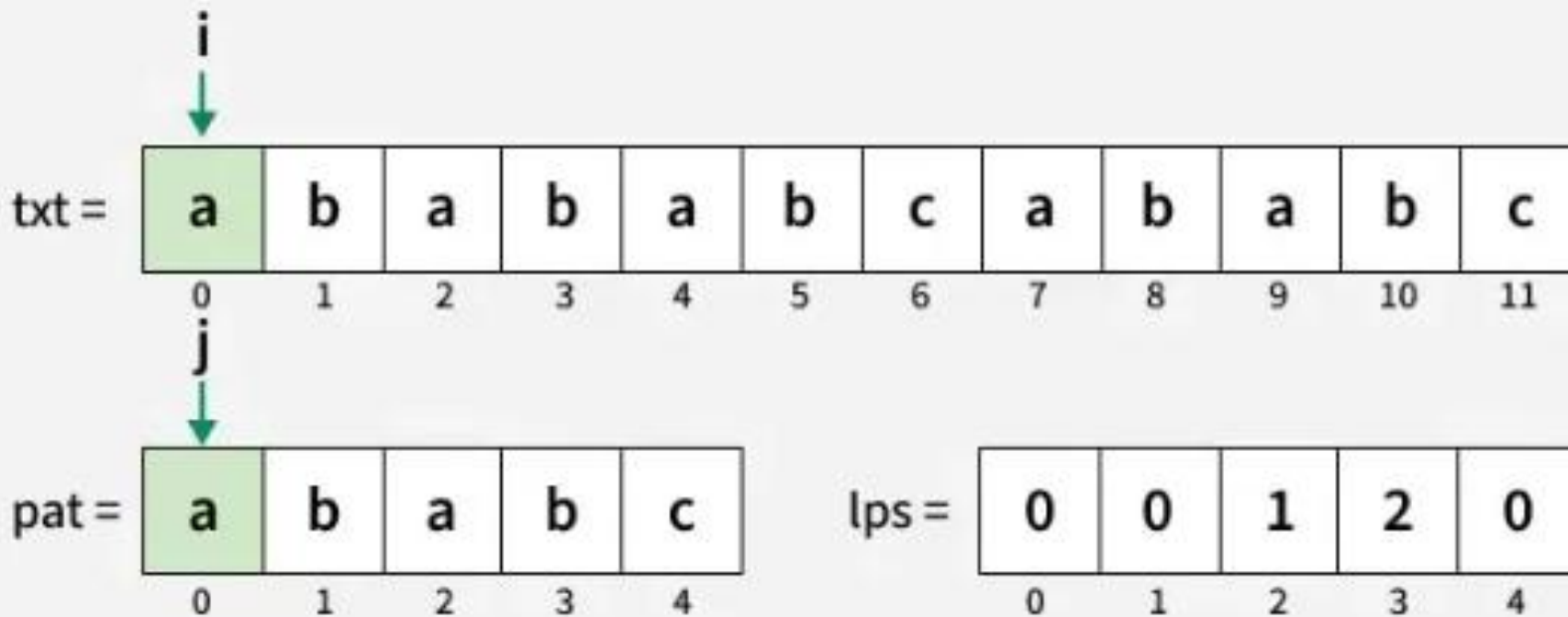


Result = []

Working of KMP

02
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

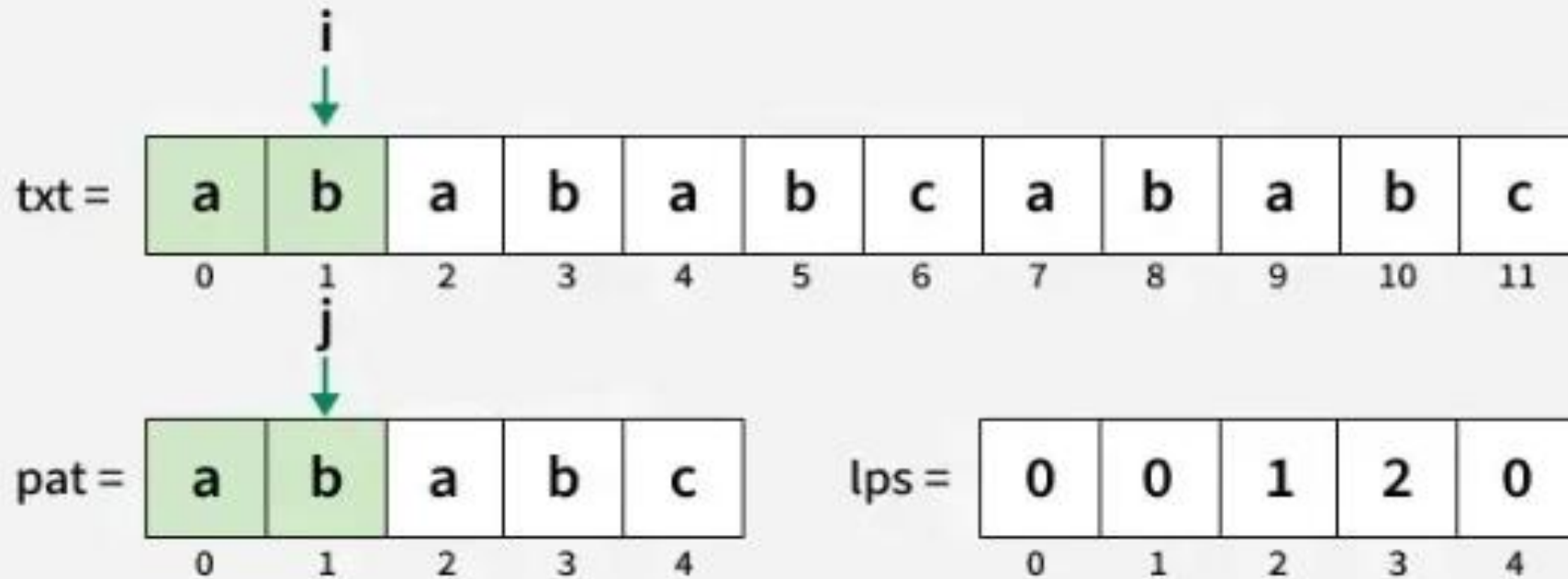


Result = []

Working of KMP

03
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

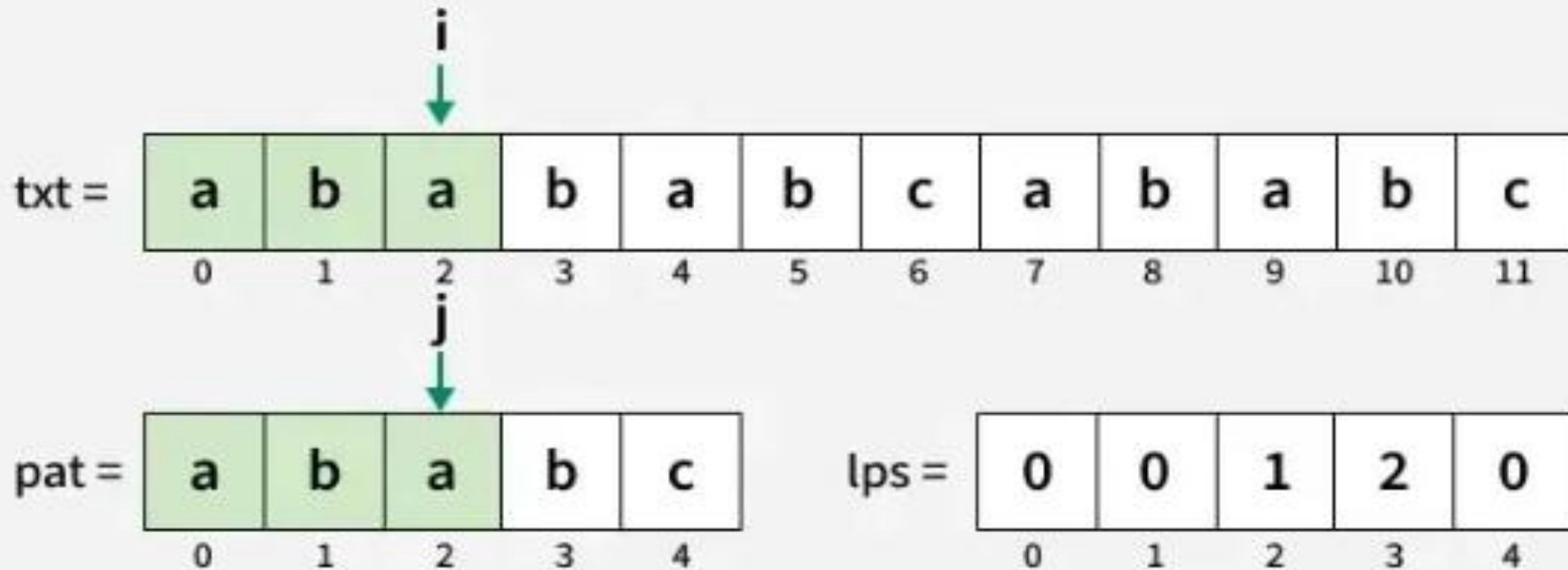


Result = []

Working of KMP

04
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

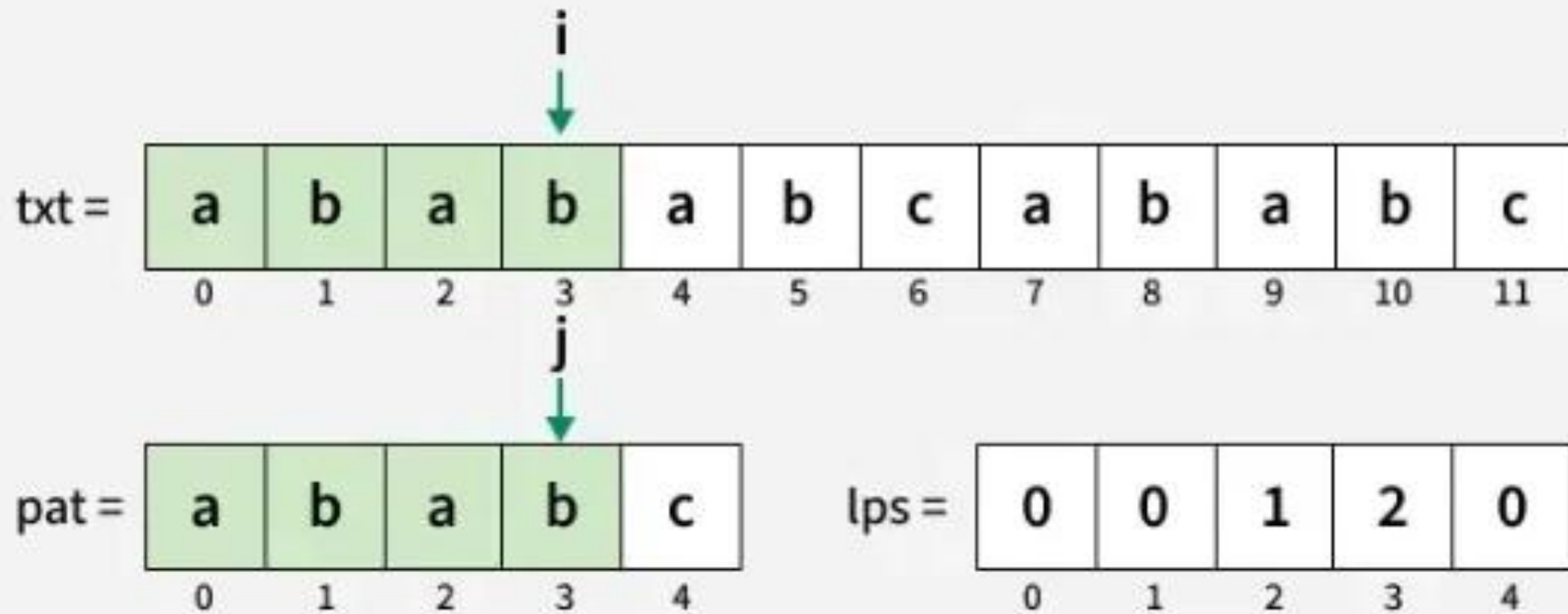


Result = []

Working of KMP

05
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

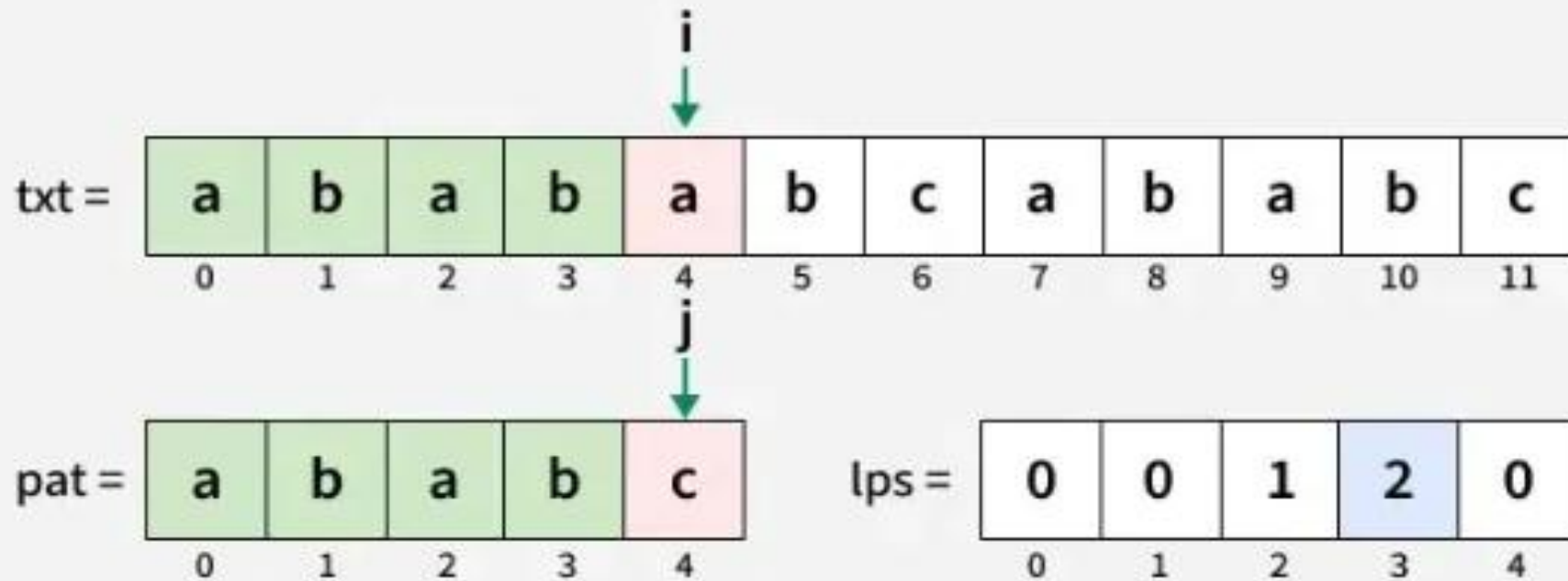


Result = []

Working of KMP

06
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ do not match, update j to $\text{lps}[j-1] = 2$

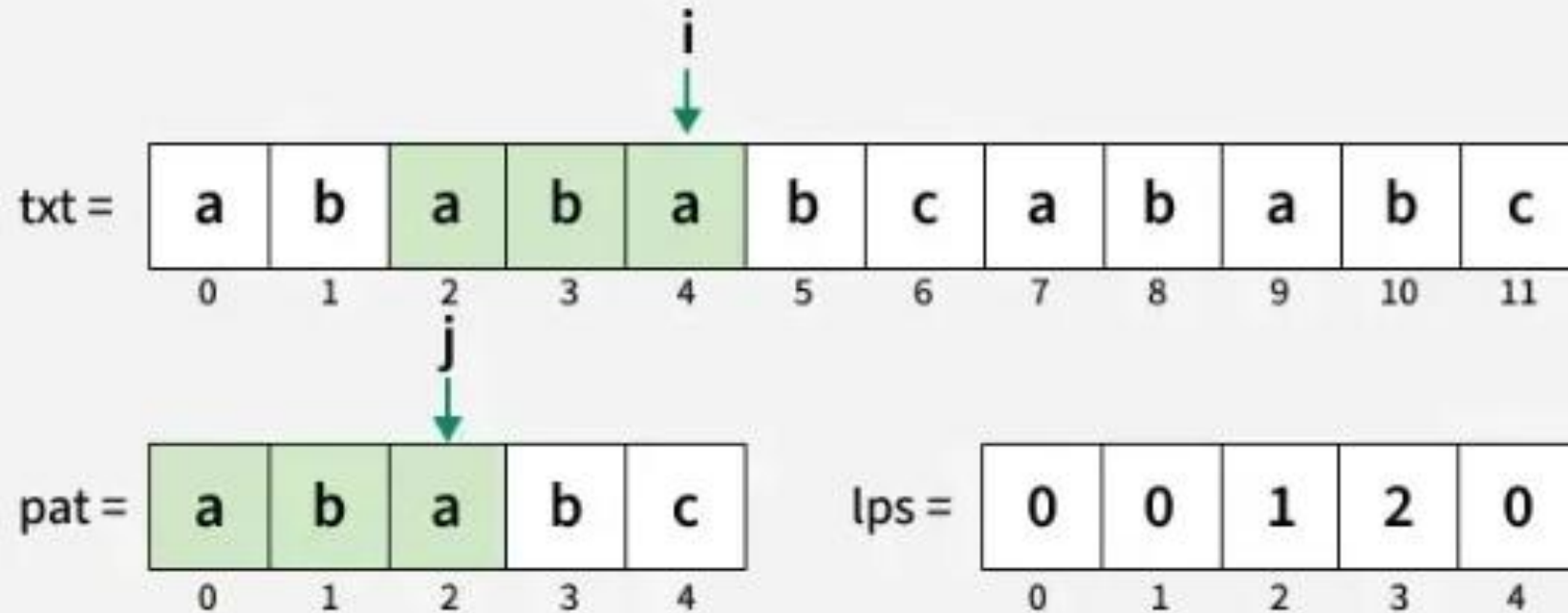


Result = []

Working of KMP

07
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

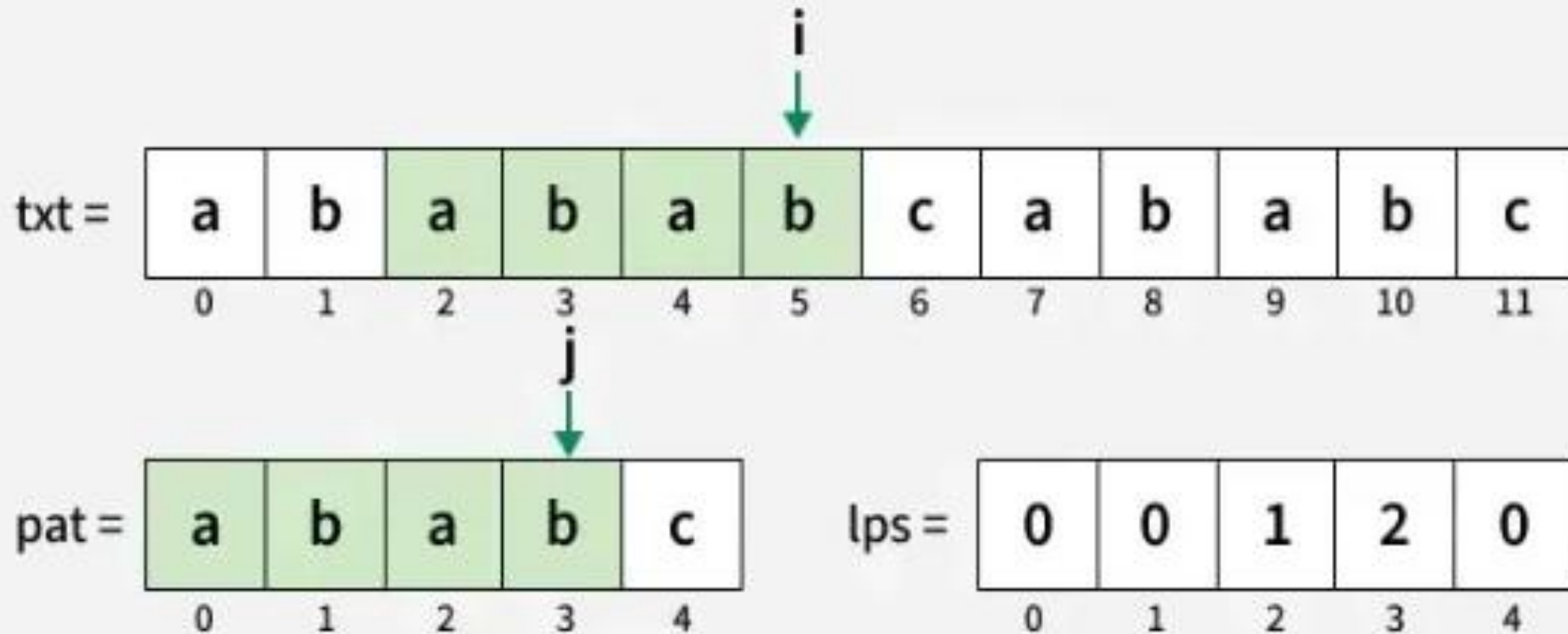


Result = []

Working of KMP

08
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

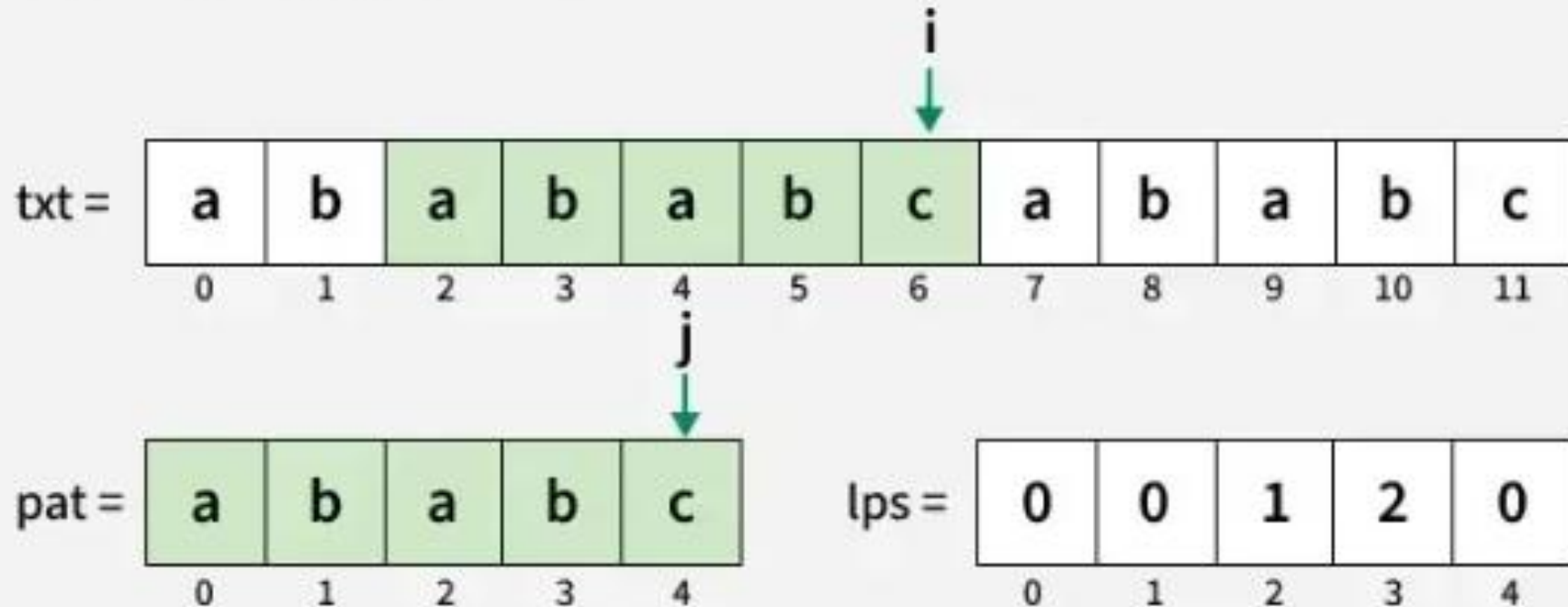


Result = []

Working of KMP

09
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .
Entire pattern has been traversed add the starting index ($i-j$) in the result
And update j to $\text{lps}[j-1] = 0$.

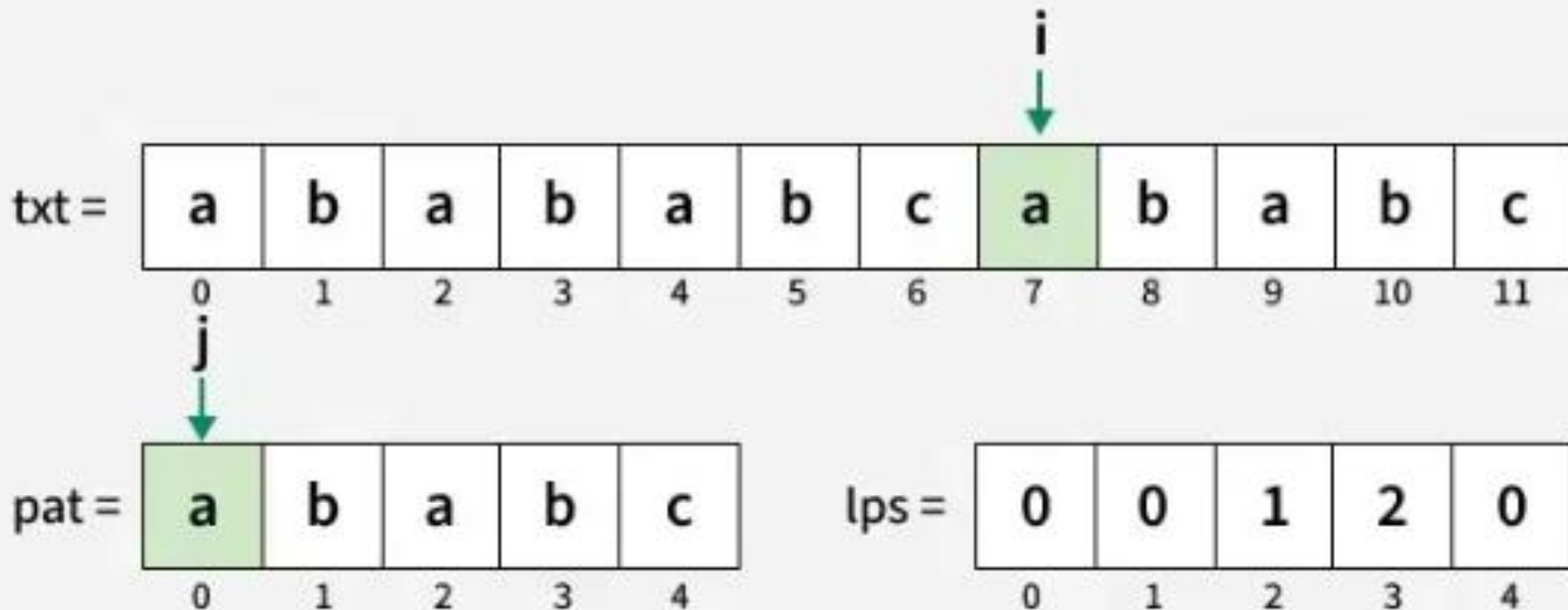


Result = [2]

Working of KMP

10
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .



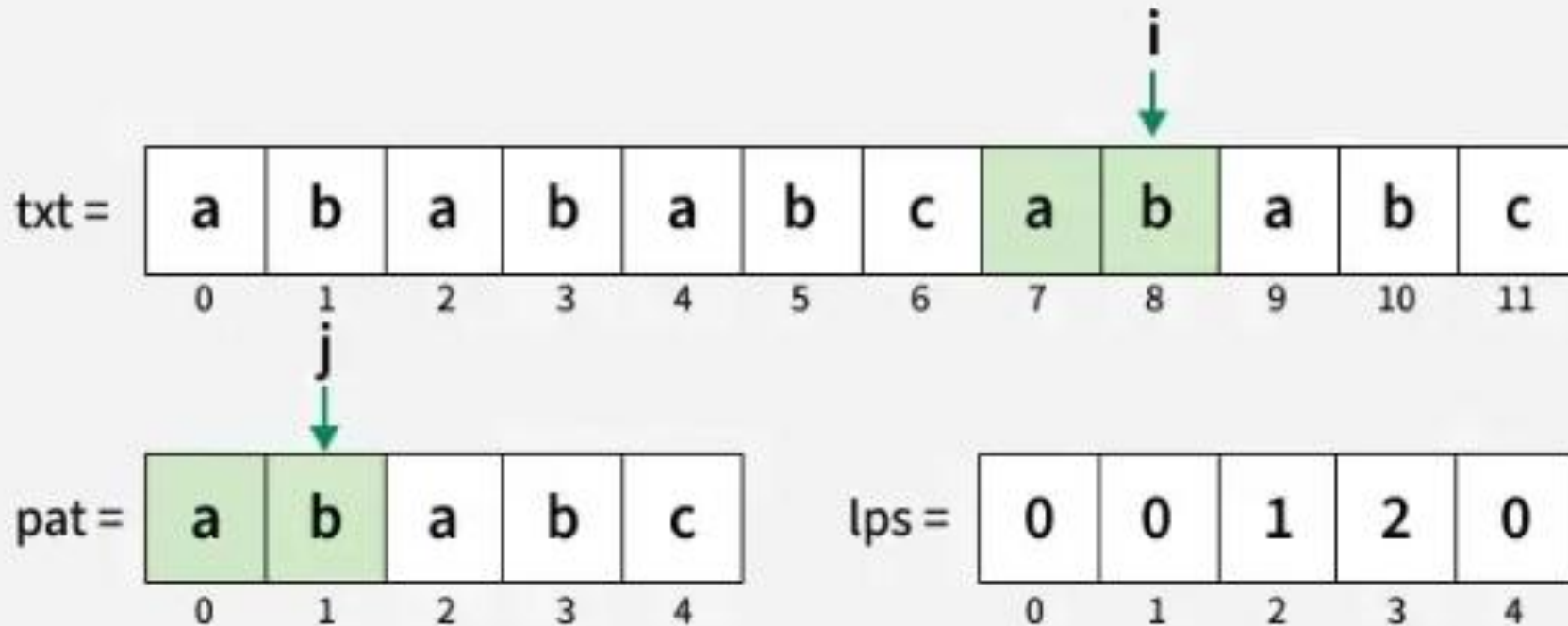
Result = [2]

Working of KMP

11

Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

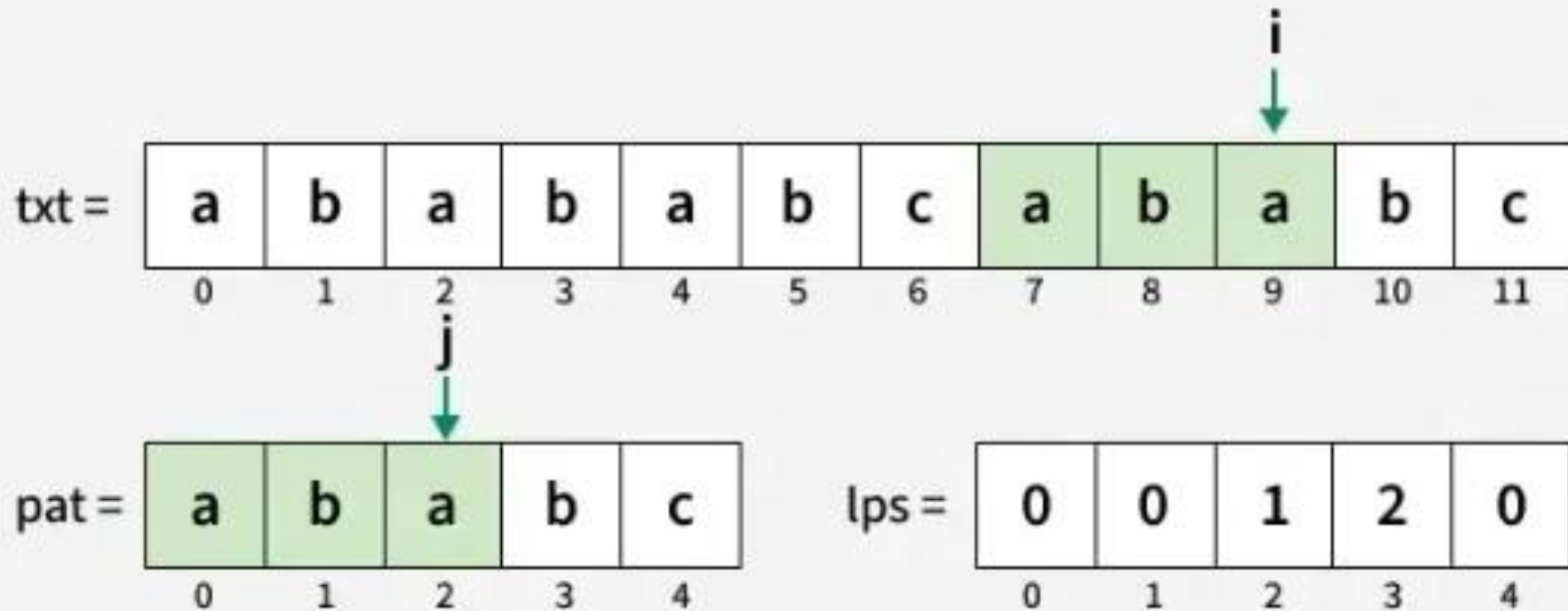


Result = [2]

Working of KMP

12
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

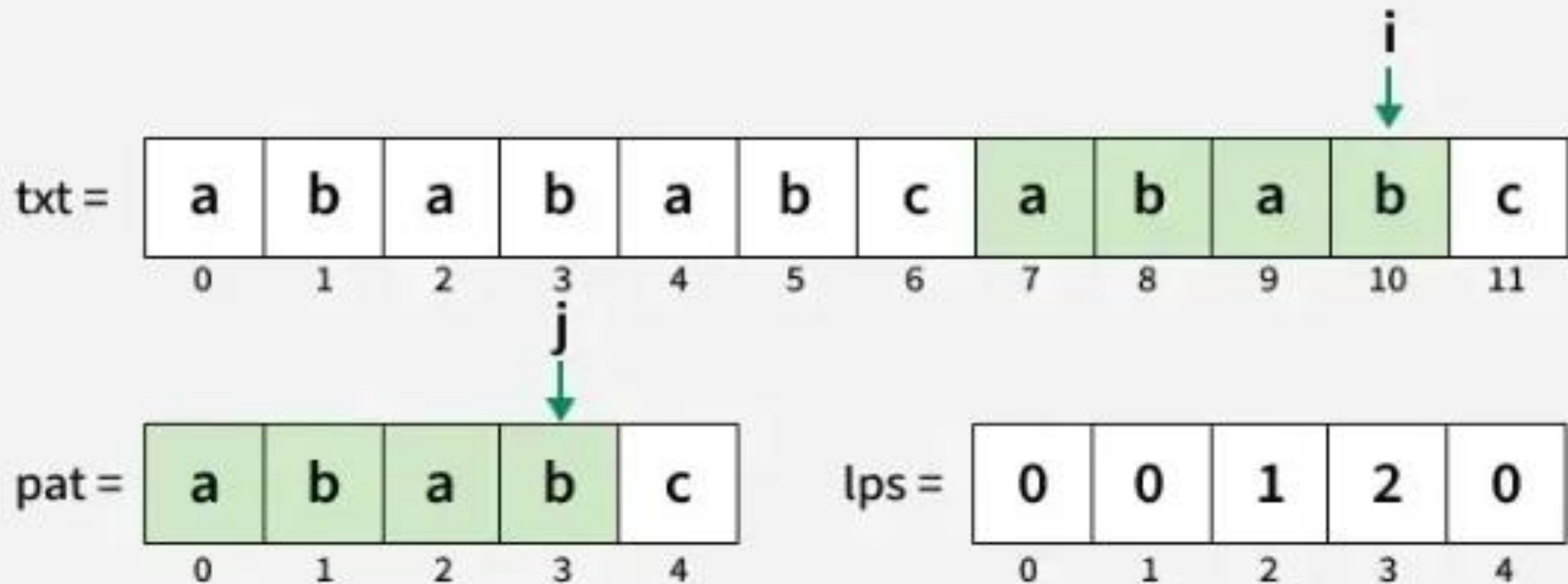


Result = [2]

Working of KMP

13
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .

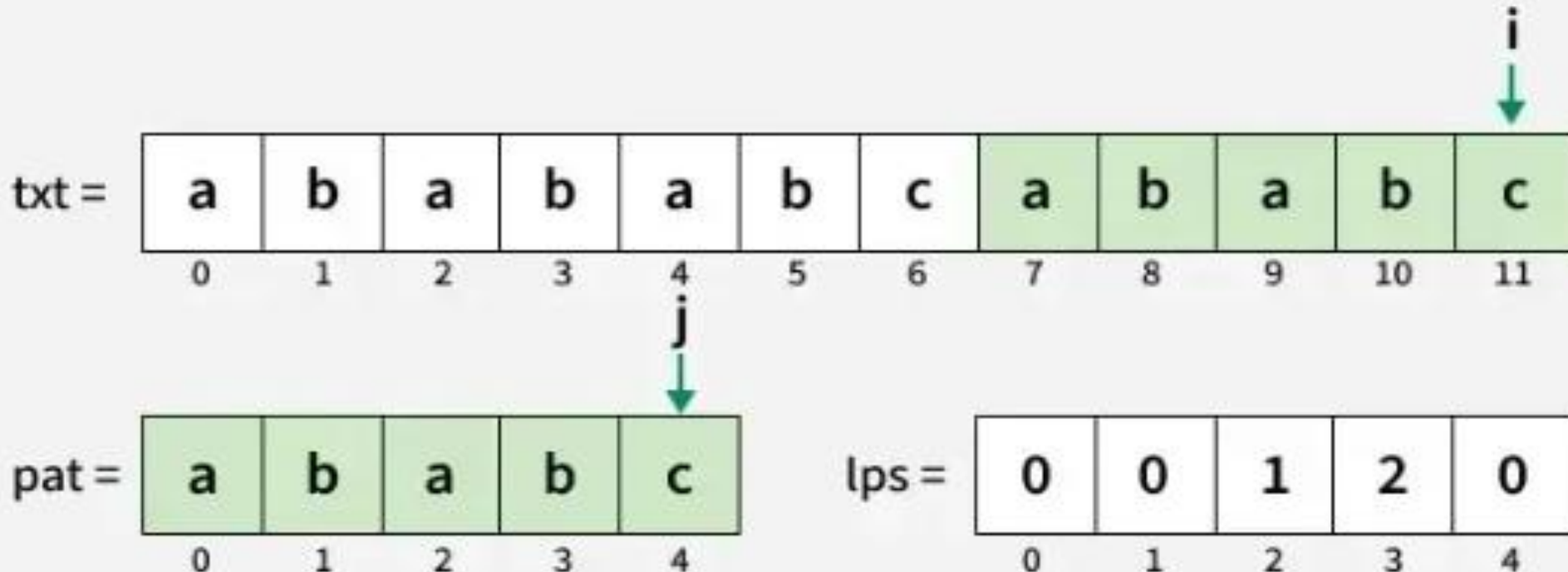


Result = [2]

Working of KMP

14
Step

Since $\text{txt}[i]$ and $\text{pat}[j]$ match, increment both i and j .
Entire pattern has been traversed add the starting index ($i-j$) in the result.



Result = [2, 7]

How KMP Algorithm works

• Text : a b a b c a b c a b a b a b d

'i' - S [index] ->

'j' - P [index] -> 0

Pattern

a	b	a	b	d
0	0	1	2	0

If 'j' reaches the end of pattern & 'i' reaches the n+1, then we can say that the given pattern is present in the given string.

The Time Complexity of KMP is

m + n
 ↑ ↑
 Table + Search

$O(m+n)$ – This is the one of the Fastest algorithm in Pattern Matching Algorithm.

The Basic Idea in KMP is If a substring of a pattern is again appeared in the pattern, then no need to compare again and again. In that way, we can reduce the number of comparisons.

Here, we are not backtracking the 'i', only 'j' is backtracking.

KMP Algorithm

Step 1: Read the Text (T) and Pattern (P).

Step 2: Construct the LPS (Longest Prefix Suffix) array for the pattern.

Step 3: Initialize two indices:

$i = 0$ (for the Text)

$j = 0$ (for the Pattern)

Step 4: Compare the current characters of the Text and Pattern.

- If the characters match:
 - Move both indices (i and j) to the next character.
- If all characters of the pattern are matched ($j = m$):
 - Display "Pattern Found".
 - Continue searching using the LPS value.
- If a mismatch occurs:
 - If j is not 0, set $j = \text{LPS}[j - 1]$ and continue matching.
 - Otherwise, move to the next character in the text ($i = i + 1$).

Step 5: Repeat the above steps until the entire text has been searched.

Step 6: Stop.

Constructing the LPS Array

```
function computeLPS(P, m):  
    lps[0] = 0  
    len = 0      # length of previous longest prefix-suffix  
    i = 1  
    while i < m:  
        if P[i] == P[len]:  
            len = len + 1  
            lps[i] = len  
            i = i + 1  
        else:  
            if len != 0:  
                len = lps[len - 1] # fall back, do NOT advance i  
            else:  
                lps[i] = 0  
                i = i + 1  
    return lps
```

```
function KMP(T, n, P, m):  
    lps = computeLPS(P, m)  
    i = 0      # index into text T  
    j = 0      # index into pattern P  
    while i < n:  
        if T[i] == P[j]:  
            i = i + 1  
            j = j + 1  
            if j == m:  
                report match at index (i - j)  
                j = lps[j - 1] # continue searching for more matches  
        else:  
            if j != 0:  
                j = lps[j - 1] # shift pattern, reuse matched prefix  
            else:  
                i = i + 1  
    # no more matches
```

//JAVA program for implementation of KMP pattern searching algorithm

```
class KMPString {  
    void KMPSearch(String pat, String txt){  
        int M = pat.length();  
        int N = txt.length();  
        int lps[] = new int[M];  
        int j = 0;  
        computeLPSArray(pat, M, lps);  
        int i = 0;  
        while (i < N) {  
            if (pat.charAt(j) == txt.charAt(i)) {  
                j++; i++; }  
            if (j == M) {  
                System.out.println("Found pattern " + "at index " + (i - j));  
                j = lps[j - 1];  
            }  
            else if (i < N && pat.charAt(j) != txt.charAt(i))  
            {  
                if (j != 0)  
                    j = lps[j - 1];  
                else  
                    i = i + 1;  
            } //else if close  
        } //while close  
    } //function close  
}
```

```
void computeLPSArray(String pat, int M, int lps[]){  
    int len = 0;  
    int i = 1;  
    lps[0] = 0;  
    while (i < M) {  
        if (pat.charAt(i) == pat.charAt(len)) {  
            len++; lps[i] = len; i++; }  
        else  
        {  
            if (len != 0)  
                len = lps[len - 1];  
            else  
            {  
                lps[i] = len;  
                i++;  
            }  
        }  
    }  
}
```

```
public static void main(String args[])  
{  
    String txt = "ABABDABACDABABCABAB";  
    String pat = "ABA";  
    new KMPString().KMPSearch(pat, txt);  
} //KMPSgtring class close
```

Output:

```
Found pattern at index 0  
Found pattern at index 5  
Found pattern at index 10  
Found pattern at index 15
```

Comparison Between Naïve and KMP

Feature	Naïve Pattern Matching	KMP Algorithm
Pattern Preprocessing	No	Yes (LPS array)
Uses Failure Function	No	Yes
Repeated Comparisons	Yes	No
Restart After Mismatch	From beginning	From LPS value
Worst-Case Time	$O(n \times m)$	$O(n + m)$
Extra Space	$O(1)$	$O(m)$
Suitable For	Small texts	Large texts and repeated searches

Applications of KMP

- ✓ Search engines
- ✓ Text editors (Find and Replace)
- ✓ DNA sequence analysis
- ✓ Bioinformatics
- ✓ Plagiarism detection
- ✓ Antivirus signature matching
- ✓ Log file analysis
- ✓ Network intrusion detection
- ✓ Digital libraries
- ✓ Information retrieval systems

Thankyou